

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

```
# Load the dataset
```

```
df = pd.read_csv('Tata Motors Stock Analysis Prediction(1995-2025).csv')
df
```

	Unnamed: 0	Date	Symbol	Open	High	Low	Close
0	223	02-01-1995	TATAMOTORS	517.50	524.70	517.50	524.20
1	224	03-01-1995	TATAMOTORS	518.00	520.00	515.00	515.00
2	225	04-01-1995	TATAMOTORS	515.00	515.00	515.00	515.00
3	226	05-01-1995	TATAMOTORS	480.00	480.00	480.00	480.00
4	227	06-01-1995	TATAMOTORS	495.00	501.00	495.00	498.70
...	...	...	...	...	...	...	...
7799	7660	12-08-2025	TATAMOTORS	651.10	660.95	650.80	654.10
7800	7661	13-08-2025	TATAMOTORS	657.20	667.00	656.65	663.75
7801	7662	14-08-2025	TATAMOTORS	666.00	666.55	657.70	664.60
7802	7663	18-08-2025	TATAMOTORS	677.00	685.00	672.90	676.00
7803	7664	19-08-2025	TATAMOTORS	676.75	703.35	673.10	700.25
	PrevClose	Volume	Turnover	VWAP	Trades	Daily_Return_%	
0	522.00	1100	5.735300e+05	521.39	NaN	0.421456	\
1	524.20	1500	7.754500e+05	516.97	NaN	1.755055	-
2	515.00	300	1.545000e+05	515.00	NaN	0.000000	
3	515.00	50	2.400000e+04	480.00	NaN	6.796117	-
4	480.00	200	9.955000e+04	497.75	NaN	3.895833	
...	...	...	...	...	...	...	
...							

```

7799      653.75  11212223  7.348231e+09  655.38  223425.0
0.053537
7800      654.10   7557556  5.016113e+09  663.72  133862.0
1.475310
7801      663.75   6250494  4.140530e+09  662.43  115002.0
0.128060
7802      664.60  14226172  9.655303e+09  678.70  165366.0
1.715317
7803      676.00  19440384  1.351303e+10  695.10  282726.0
3.587278

```

```

      MA_20  MA_50
0      NaN   NaN
1      NaN   NaN
2      NaN   NaN
3      NaN   NaN
4      NaN   NaN
...      ...   ...
7799  669.3700 683.496
7800  668.6075 682.568
7801  667.7525 681.640
7802  667.5400 680.804
7803  668.1800 680.164

```

[7804 rows x 15 columns]

df.head() *# View the first rows of the dataset*

```

      Unnamed: 0      Date      Symbol  Open  High  Low  Close
PrevClose
0      223  02-01-1995  TATAMOTORS  517.5  524.7  517.5  524.2
522.0  \
1      224  03-01-1995  TATAMOTORS  518.0  520.0  515.0  515.0
524.2
2      225  04-01-1995  TATAMOTORS  515.0  515.0  515.0  515.0
515.0
3      226  05-01-1995  TATAMOTORS  480.0  480.0  480.0  480.0
515.0
4      227  06-01-1995  TATAMOTORS  495.0  501.0  495.0  498.7
480.0

```

```

      Volume  Turnover  VWAP  Trades  Daily_Return_%  MA_20  MA_50
0      1100  573530.0  521.39    NaN      0.421456    NaN   NaN
1      1500  775450.0  516.97    NaN     -1.755055    NaN   NaN
2       300  154500.0  515.00    NaN      0.000000    NaN   NaN
3        50   24000.0  480.00    NaN     -6.796117    NaN   NaN
4       200   99550.0  497.75    NaN      3.895833    NaN   NaN

```

df.tail() *# View the last rows of the dataset*

Unnamed: 0	Date	Symbol	Open	High	Low	Close
7799	12-08-2025	TATAMOTORS	651.10	660.95	650.80	654.10
7800	13-08-2025	TATAMOTORS	657.20	667.00	656.65	663.75
7801	14-08-2025	TATAMOTORS	666.00	666.55	657.70	664.60
7802	18-08-2025	TATAMOTORS	677.00	685.00	672.90	676.00
7803	19-08-2025	TATAMOTORS	676.75	703.35	673.10	700.25

PrevClose	Volume	Turnover	VWAP	Trades	Daily_Return_%
653.75	11212223	7.348231e+09	655.38	223425.0	0.053537
654.10	7557556	5.016113e+09	663.72	133862.0	1.475310
663.75	6250494	4.140530e+09	662.43	115002.0	0.128060
664.60	14226172	9.655303e+09	678.70	165366.0	1.715317
676.00	19440384	1.351303e+10	695.10	282726.0	3.587278

MA_20	MA_50
669.3700	683.496
668.6075	682.568
667.7525	681.640
667.5400	680.804
668.1800	680.164

```
df.shape
```

Returns a tuple with number of rows and columns

```
(7804, 15)
```

```
df.columns
```

Lists column names.

```
Index(['Unnamed: 0', 'Date', 'Symbol', 'Open', 'High', 'Low', 'Close',
       'PrevClose', 'Volume', 'Turnover', 'VWAP', 'Trades',
       'Daily_Return_%',
       'MA_20', 'MA_50'],
      dtype='object')
```

```
df.info()
```

Gives summary including data types and non-null values.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7804 entries, 0 to 7803
```

```
Data columns (total 15 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Unnamed: 0           7804 non-null   int64
1   Date                 7804 non-null   object
2   Symbol               7804 non-null   object
3   Open                 7804 non-null   float64
4   High                 7804 non-null   float64
5   Low                  7804 non-null   float64
6   Close                7804 non-null   float64
7   PrevClose            7804 non-null   float64
8   Volume               7804 non-null   int64
9   Turnover              7804 non-null   float64
10  VWAP                 7804 non-null   float64
11  Trades               3617 non-null   float64
12  Daily_Return_%       7804 non-null   float64
13  MA_20                7785 non-null   float64
14  MA_50                7755 non-null   float64
```

```
dtypes: float64(11), int64(2), object(2)
```

```
memory usage: 914.7+ KB
```

```
df.describe()           # Statistical summary of numerical columns.
```

	Unnamed: 0	Open	High	Low	Close
count	7804.000000	7804.000000	7804.000000	7804.000000	7804.000000
\					
mean	3901.500000	426.908105	433.624885	419.198526	426.156516
std	2252.965083	260.410952	263.730360	256.249596	259.926045
min	0.000000	58.000000	60.700000	57.550000	58.800000
25%	1950.750000	215.950000	220.187500	209.437500	215.375000
50%	3901.500000	399.000000	404.525000	392.175000	397.800000
75%	5852.250000	535.000000	544.000000	526.262500	535.312500
max	7803.000000	1361.000000	1382.000000	1347.000000	1365.150000

	PrevClose	Volume	Turnover	VWAP
Trades				
count	7804.000000	7.804000e+03	7.804000e+03	7804.000000
3.617000e+03 \				
mean	426.042984	1.008443e+07	3.360758e+09	426.517865
1.641804e+05				
std	259.923227	1.998049e+07	5.490162e+09	259.954308
1.309842e+05				
min	0.000000	5.000000e+01	2.400000e+04	59.240000

```

3.434000e+03
25%      215.375000  1.365342e+06  4.126361e+08  215.487500
8.477600e+04
50%      397.800000  4.114132e+06  1.913436e+09  398.320000
1.263780e+05
75%      535.312500  9.874223e+06  3.942626e+09  535.530000
1.993340e+05
max      1365.150000  3.905778e+08  9.853175e+10  1362.150000
1.517162e+06

```

	Daily_Return_%	MA_20	MA_50
count	7804.000000	7785.000000	7755.000000
mean	inf	425.784914	425.176591
std	NaN	258.907945	257.163936
min	-80.866780	64.885000	69.538000
25%	-1.418222	215.867500	219.790000
50%	0.013436	398.915000	401.483000
75%	1.490380	533.560000	525.136500
max	inf	1316.080000	1270.157000

```
df.dtypes                                # Data types of each column.
```

```

Unnamed: 0      int64
Date            object
Symbol          object
Open            float64
High            float64
Low             float64
Close           float64
PrevClose       float64
Volume          int64
Turnover        float64
VWAP            float64
Trades          float64
Daily_Return_%  float64
MA_20           float64
MA_50           float64
dtype: object

```

```
# Data Cleaning and Preprocessing
```

```
df.isnull().sum()                                # Counts missing values per column
```

```

Unnamed: 0      0
Date            0
Symbol          0
Open            0
High            0
Low             0
Close           0
PrevClose       0

```

```
Volume      0
Turnover    0
VWAP        0
Trades      4187
Daily_Return_%  0
MA_20       19
MA_50       49
dtype: int64
```

```
df = df.drop(columns=['Unnamed: 0'])
```

```
df.isnull().sum()           # Verify missing values are removed
```

```
Date      0
Symbol    0
Open      0
High      0
Low       0
Close     0
PrevClose 0
Volume    0
Turnover  0
VWAP      0
Trades    4187
Daily_Return_%  0
MA_20     19
MA_50     49
dtype: int64
```

```
df['Trades'] = df['Trades'].ffill().bfill()
df['MA_20'] = df['Close'].rolling(20, min_periods=1).mean()
df['MA_50'] = df['Close'].rolling(50, min_periods=1).mean()
```

4. Data Exploration

```
print(df['Symbol'].value_counts())           # Counts unique values.
```

```
Symbol
TATAMOTORS    7804
Name: count, dtype: int64
```

```
print("Unique Symbols:", df['Symbol'].nunique())           # Returns number
of unique values
```

```
Unique Symbols: 1
```

```
df.rename(columns={'Unnamed: 0': 'Index'}, inplace=True)
```

```
print("Correlation Matrix:\n", df.corr(numeric_only=True))
# Correlation matrix.
```

Correlation Matrix:

	Open	High	Low	Close	PrevClose
Volume					
Open	1.000000	0.999562	0.999396	0.999109	0.999009
0.182193 \					
High	0.999562	1.000000	0.999272	0.999605	0.998745
0.179688					
Low	0.999396	0.999272	1.000000	0.999553	0.998504
0.182901					
Close	0.999109	0.999605	0.999553	1.000000	0.998335
0.180607					
PrevClose	0.999009	0.998745	0.998504	0.998335	1.000000
0.182716					
Volume	-0.182193	-0.179688	-0.182901	-0.180607	-0.182716
1.000000					
Turnover	0.137173	0.140232	0.136580	0.139185	0.136211
0.801482					
VWAP	0.999490	0.999787	0.999711	0.999858	0.998714
0.181084					
Trades	0.068155	0.069376	0.066728	0.067984	0.068199
0.768628					
Daily_Return_%	-0.008437	0.005174	0.006829	0.018268	-0.025179
0.091408					
MA_20	0.992191	0.992097	0.991500	0.991550	0.992546
0.189918					
MA_50	0.975462	0.975350	0.974627	0.974673	0.976044
0.196314					
	Turnover	VWAP	Trades	Daily_Return_%	MA_20
Open	0.137173	0.999490	0.068155	-0.008437	0.992191
\					
High	0.140232	0.999787	0.069376	0.005174	0.992097
Low	0.136580	0.999711	0.066728	0.006829	0.991500
Close	0.139185	0.999858	0.067984	0.018268	0.991550
PrevClose	0.136211	0.998714	0.068199	-0.025179	0.992546
Volume	0.801482	-0.181084	0.768628	0.091408	-0.189918
Turnover	1.000000	0.138783	0.926621	0.088657	0.125056
VWAP	0.138783	1.000000	0.068043	0.009253	0.991816
Trades	0.926621	0.068043	1.000000	0.036066	0.064080
Daily_Return_%	0.088657	0.009253	0.036066	1.000000	-0.024247
MA_20	0.125056	0.991816	0.064080	-0.024247	1.000000

MA_50	0.110991	0.974917	0.058863	-0.028930	0.990647
-------	----------	----------	----------	-----------	----------

	MA_50
Open	0.975462
High	0.975350
Low	0.974627
Close	0.974673
PrevClose	0.976044
Volume	-0.196314
Turnover	0.110991
VWAP	0.974917
Trades	0.058863
Daily_Return_%	-0.028930
MA_20	0.990647
MA_50	1.000000

```
# Group by Symbol                                # Mean of groups by a column.
```

```
print(df.groupby('Symbol')['Close'].mean())
```

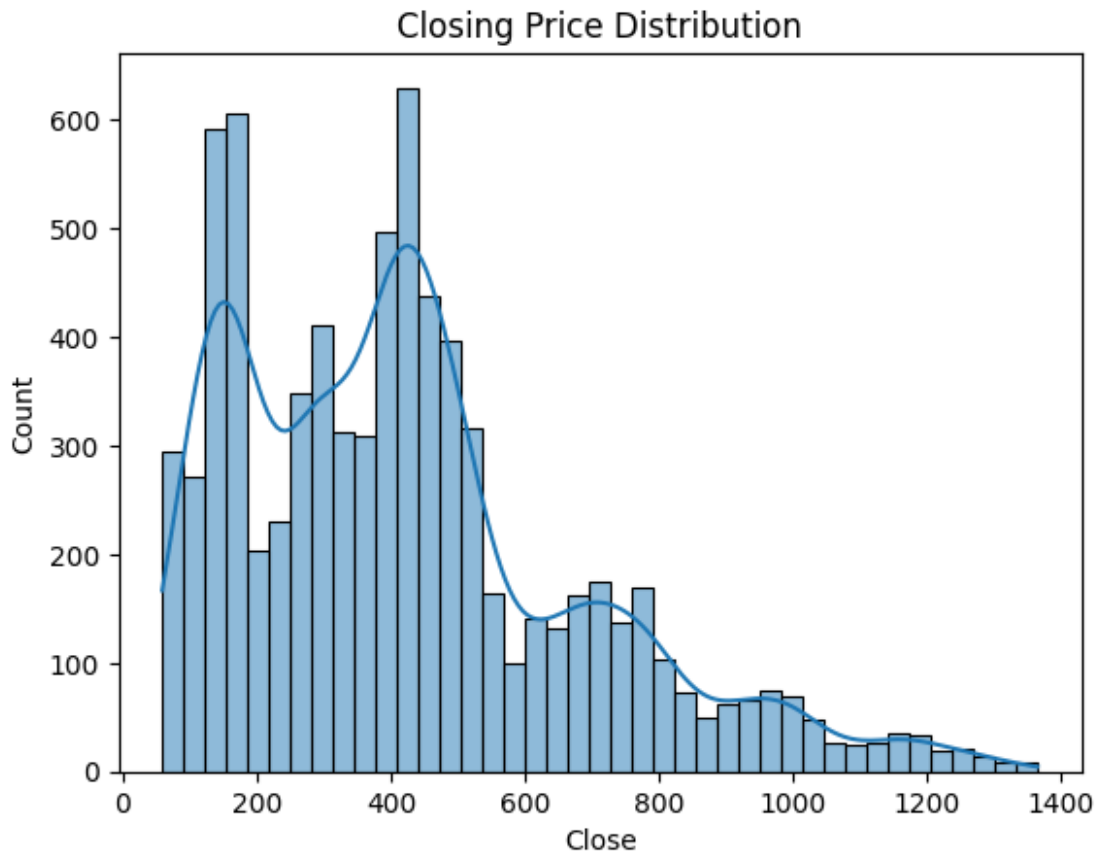
```
Symbol
TATAMOTORS    426.156516
Name: Close, dtype: float64
```

```
# Data Visualization
```

```
# How are closing prices distributed over time?
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
sns.histplot(df['Close'], kde=True)
plt.title("Closing Price Distribution")
plt.show()
```

Insights:

1.The multi-modal distribution (peaks at 150–250, 350–500, and 900–1050) indicates the asset transitioned through distinct market regimes—from an early low-price phase to mid-range consolidation and later high-price expansion, typical of growth or speculative assets.

2.The 300–500 range represents the asset's fair value zone, where price acceptance, liquidity, and consolidation are highest.

3.The long right tail indicates positive skewness, meaning rare but sharp upside spikes, more frequent mild declines, and a mean inflated above the median.

4.The wide 100–1100+ range (10x spread) indicates high volatility, suggesting the asset is unstable and highly sensitive to sentiment, external shocks, and news.

5.The multi-peaked, skewed distribution is non-normal, meaning standard models may underestimate risk, so regime-switching, log transformations, or quantile-based methods are more appropriate.

Are there outliers in closing prices?

```
sns.boxplot(y=df['Close'])  
plt.title("Outliers in Closing Price")  
plt.show()
```



```
# Insights:  
# 1. Median around ~400 → Indicates the typical closing price lies near  
the middle of the 300–500 range.  
# 2. Interquartile Range (IQR) ~270 to ~500 → Most trading activity is  
concentrated here, reinforcing this as the core value zone.  
# 3. Wide overall spread → Prices range from ~70 to ~1150+, confirming  
high volatility.  
# 4. Strong upper outliers → Numerous points above ~850 show frequent  
extreme upward spikes.  
# 5. Few lower outliers → Limited extreme downside values suggest less  
severe negative shocks.  
# 6. Positive skewness → Longer upper whisker and many high-end  
outliers indicate asymmetry toward higher prices.  
# 7. Potential overextension zones → Outliers above ~900 may represent  
speculative surges or unsustainable price levels.  
  
# Is there a relationship between trading volume and closing price?
```

```
sns.scatterplot(x='Volume', y='Close', data=df)  
plt.title("Volume vs Closing Price")  
plt.show()
```



Insights:

High prices occur mostly at low-to-moderate volume → The highest closing prices (~800–1150) often appear during lower volume periods, suggesting price spikes are not necessarily driven by strong participation.

Most trading activity is concentrated at lower prices → The densest cluster lies in the 100–400 range, where volume is widely scattered, indicating broad participation at lower valuation levels.

High volume does not guarantee high price → Even at very high volumes (200M–400M), prices generally remain in the ~150–350 range, showing weak upward price response to volume surges.

Heteroscedastic pattern (unequal variance) → Price variability is higher at low volumes and becomes more constrained at higher volumes, indicating inconsistent volatility behavior.

Presence of outliers in both variables → There are extreme cases of high price at low volume and very high volume at moderate prices, suggesting irregular or event-driven trading activity.

Possible retail vs institutional behavior split → Low volume with high price spikes may reflect speculative moves, while high volume with moderate prices may indicate accumulation or distribution by larger participants.

```
# Which variables are strongly related to each other?

import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# correlation
corr = df.corr(numeric_only=True)

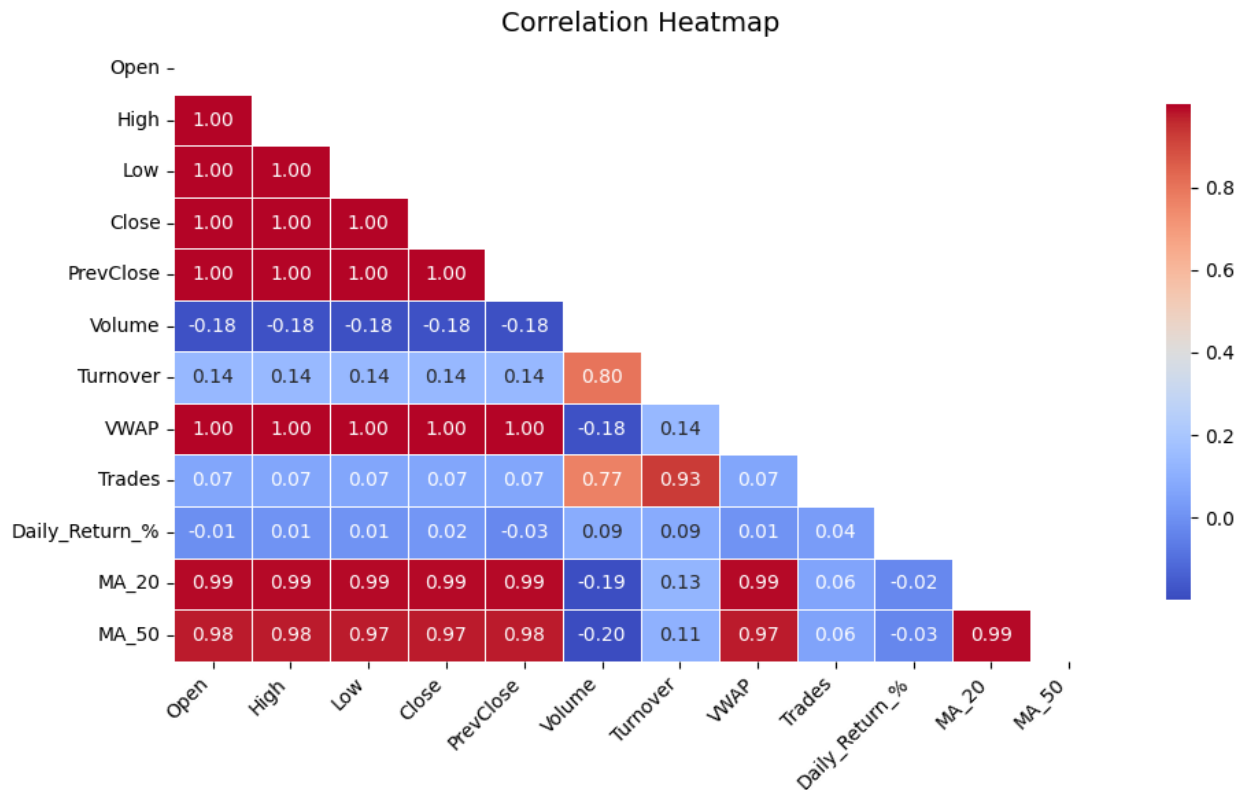
# remove redundant upper triangle
mask = np.triu(np.ones_like(corr, dtype=bool))

plt.figure(figsize=(10, 6))

sns.heatmap(
    corr,
    mask=mask,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    linewidths=0.5,
    cbar_kws={"shrink": 0.8}
)

plt.title("Correlation Heatmap", fontsize=14)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)

plt.tight_layout()
plt.show()
```

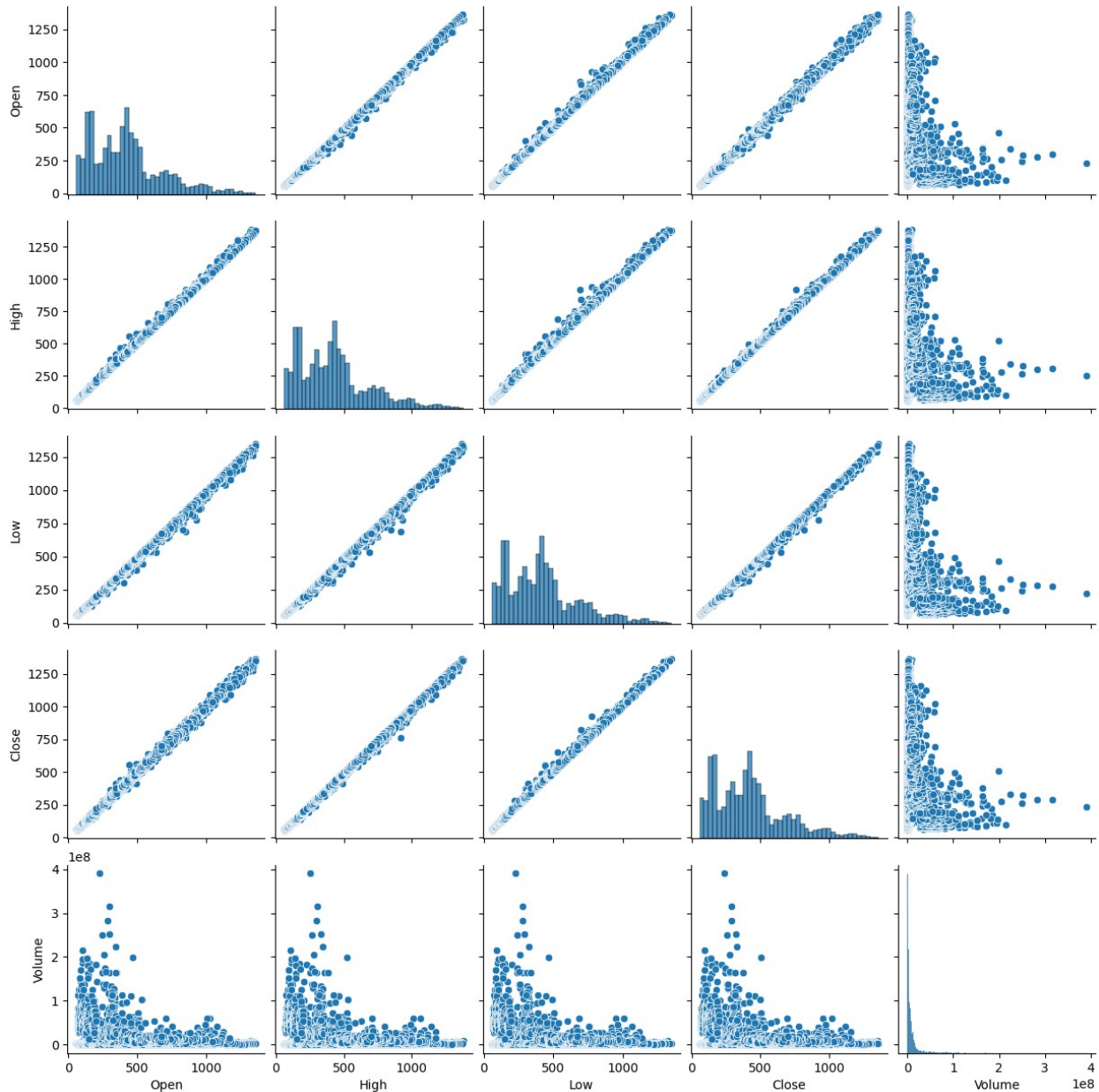


Insights:

- # 1.Price variables are highly redundant – they all move almost identically.
- # 2.Drawdown and Peak are just derived from price, not independent signals.
- # 3.Liquidity metrics (Volume, Turnover, Trades) form a separate cluster.
- # 4.Daily returns are mostly independent and highly noisy.
- # 5.Month shows no meaningful correlation – weak seasonality.
- # 6.Year shows moderate correlation – indicating an overall trend over time.

How do multiple variables relate to each other simultaneously?

```
sns.pairplot(df[['Open', 'High', 'Low', 'Close', 'Volume']])
plt.show()
```



Insights:

- # 1. The price variables (Open, High, Low, Close) show a near-perfect linear relationship, indicating they are highly redundant.
- # 2. Volume exhibits an inverse spread with price, where lower prices have higher variability in volume and higher prices show more compressed volume.
- # 3. The data displays strong heteroscedasticity, as the variance of volume changes significantly across different price levels.
- # 4. The distribution of volume is highly skewed, with a few extreme spikes dominating the overall activity.
- # 5. There is no clear linear relationship between price and volume,

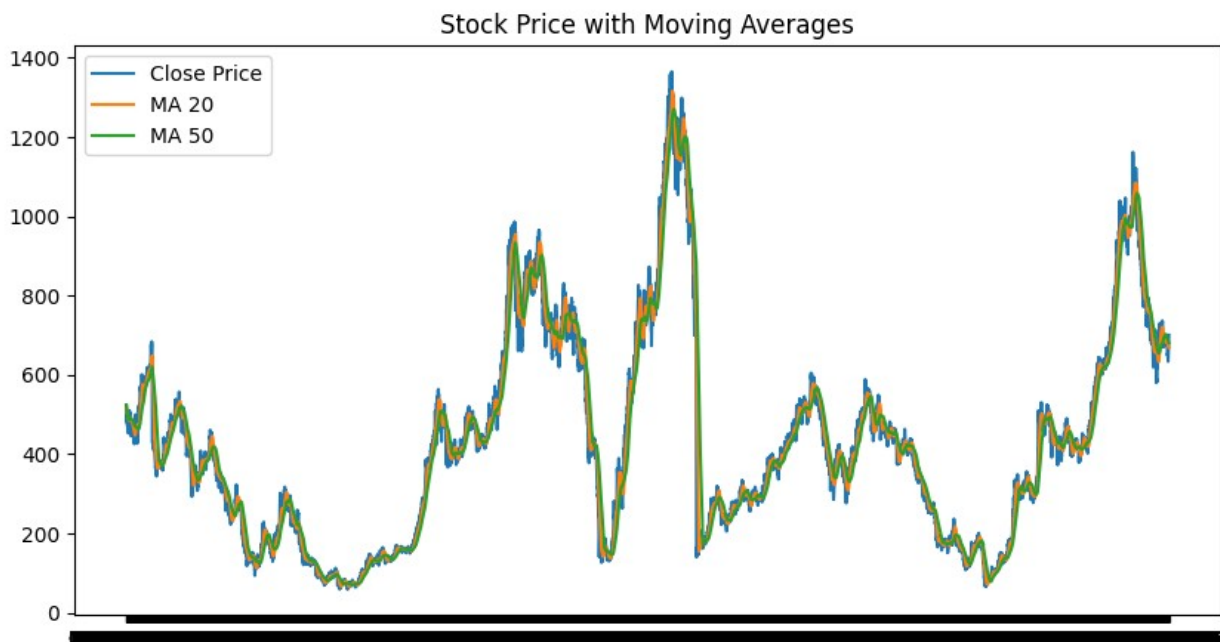
suggesting price movements are not directly driven by volume.
6. Overall, price data is repetitive while volume data is noisy but potentially more informative for deeper

What is the price trend, and how do short-term vs long-term trends compare?

```
plt.figure(figsize=(10,5))

plt.plot(df['Date'], df['Close'], label='Close Price')
plt.plot(df['Date'], df['MA_20'], label='MA 20')
plt.plot(df['Date'], df['MA_50'], label='MA 50')

plt.legend()
plt.title("Stock Price with Moving Averages")
plt.show()
```



Insights:

The price series exhibits a clear long-term upward trajectory, interrupted by distinct cyclical drawdowns and recovery phases, indicating structured trend behavior rather than randomness.
The close alignment of MA 20 and MA 50 with the price confirms strong trend persistence, with minimal lag suggesting sustained directional momentum.
Downside movements are sharp but followed by prolonged recovery periods, reflecting asymmetric market dynamics typical of accumulation and distribution phases.
Volume spikes are temporally aligned with high-volatility periods,

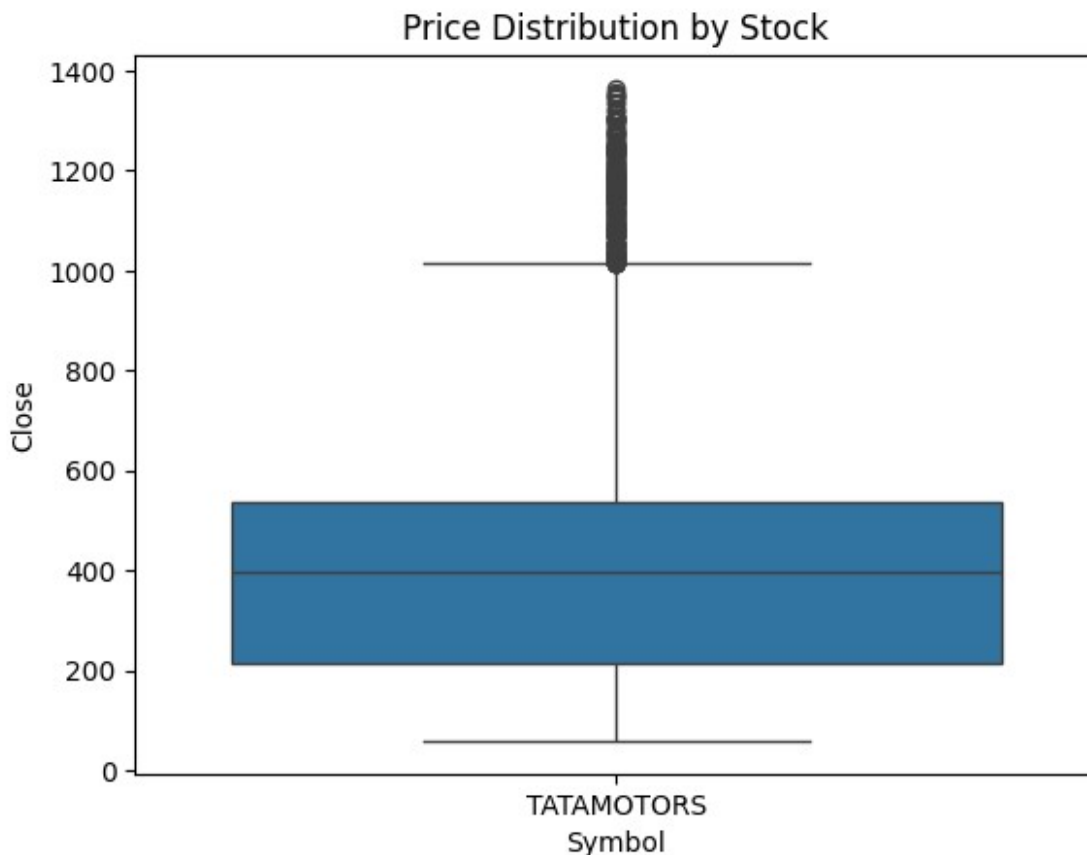
indicating that trading activity intensifies during uncertainty and major price transitions.

The recent sharp uptrend followed by a pullback suggests a potential consolidation phase, likely driven by profit-booking after an extended rally.

Overall, the asset demonstrates a trend-dominated structure with momentum-driven movements, episodic volatility clustering, and clear regime shifts over time.

How do price distributions vary across different stocks?

```
sns.boxplot(x='Symbol', y='Close', data=df)
plt.title("Price Distribution by Stock")
plt.show()
```



Insights:

The median closing price is centered around the mid-range, indicating that most trading activity is concentrated in a stable price band.

The interquartile range is relatively wide, suggesting moderate price variability over time.

A significant number of high-end outliers indicate occasional strong

bullish spikes beyond the typical range.
The upper whisker is much longer than the lower, reflecting a positively skewed distribution.
The lower range is relatively compressed, implying limited extreme downside movements compared to upside.
Overall, the distribution indicates a stable core price range with intermittent high-value surges driving positive skewn

How do stock returns vary across months and years?

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

Convert Date

```
df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)
```

Extract time features

```
df['Month'] = df['Date'].dt.month
df['Year'] = df['Date'].dt.year
```

Month labels

```
month_labels = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
```

Pivot table

```
pivot = df.pivot_table(
    values='Daily_Return_%',
    index='Year',
    columns='Month',
    aggfunc='mean'
) * 100
```

Sort months properly

```
pivot = pivot.reindex(columns=range(1,13))
pivot.columns = month_labels
```

Plot

```
plt.figure(figsize=(14,6))
sns.set_style("white")
```

```
ax = sns.heatmap(
    pivot,
    cmap='RdYlGn',           # green = positive, red = negative
    center=0,               # very important for returns
    annot=True,
    fmt=".1f",
    linewidths=0.5,
    linecolor='gray',
    cbar_kws={'label': 'Return (%)'},
```

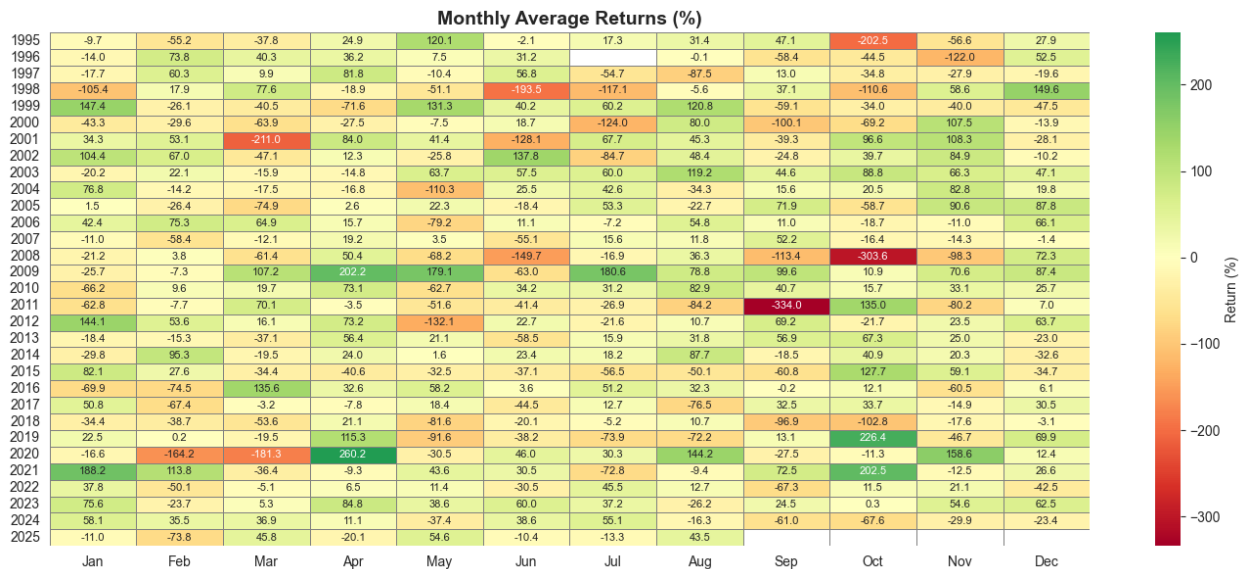
```

    annot_kws={"size":8}
)

plt.title("Monthly Average Returns (%)", fontsize=14, weight='bold')
plt.xlabel("")
plt.ylabel("")
plt.xticks(rotation=0)
plt.yticks(rotation=0)

plt.tight_layout()
plt.show()

```



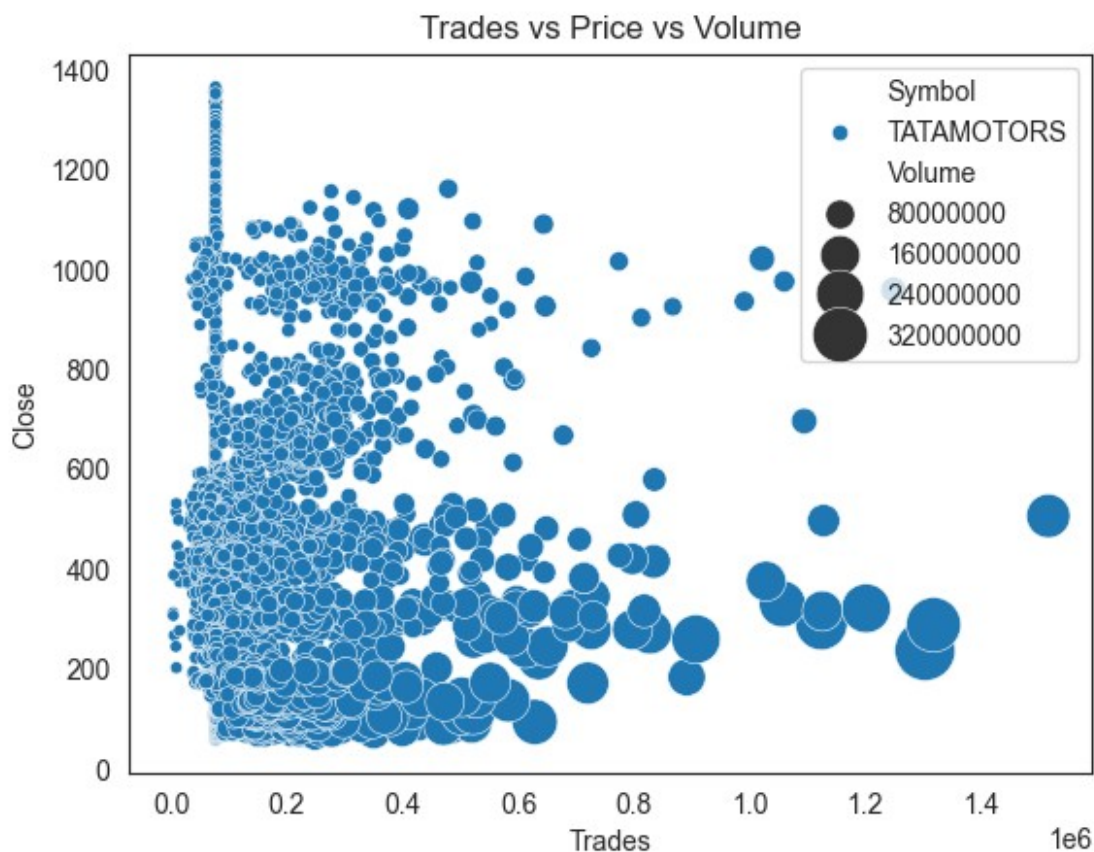
Insights:

- # 1. September 2011 stands out with an extreme loss of -333.97%, the single worst monthly return in the entire dataset, likely reflecting the global market crash and eurozone crisis impact on TATAMOTORS.
- # 2. October 2019 recorded the highest gain at +226.40%, followed closely by October 2021 (+202.52%) and April 2020 (+260.21%), suggesting October and April are historically strong months for the stock.
- # 3. Month 2 (February) and Month 3 (March) show frequent blue (negative) tones across multiple years, making early calendar months structurally weak periods with recurring losses, particularly February 2020 (-164.18) and March 2020 (-181.33).
- # 4. January shows a strong bullish bias with several high positive returns – 2012 (+144.10), 2021 (+188.23), and 2015 (+82.11) – making it one of the most consistently red (positive) months across the heatmap.
- # 5. 2020 was the most volatile year overall, displaying both the deepest losses (Feb: -164.18, Mar: -181.33) and strong recoveries

(Apr: +260.21, Aug: +144.24, Nov: +158.58), directly reflecting COVID-19 crash and subsequent stimulus-driven rebound.

How do trades, price, and volume interact across different stocks?

```
sns.scatterplot(
    x='Trades',
    y='Close',
    size='Volume',
    hue='Symbol',
    data=df,
    sizes=(20, 500)
)
plt.title("Trades vs Price vs Volume")
plt.show()
```



Insights:

Peak prices (₹900–₹1,200) occur at low trade counts (<0.4M), indicating TATAMOTORS' all-time highs were reached during relatively calm, low-participation sessions – a hallmark of institutional-driven, low-noise price discovery.

Heaviest trading days (>0.8M trades) are stuck below ₹500, revealing

that mass retail participation and high-frequency activity coincide with cheaper price levels, typical of fear-driven or speculative bottom-fishing behavior.

Largest volume bubbles (>240M) cluster between ₹200–₹400, suggesting institutional accumulation or distribution happened predominantly in this price band, making ₹200–₹400 the most contested and significant price zone historically.

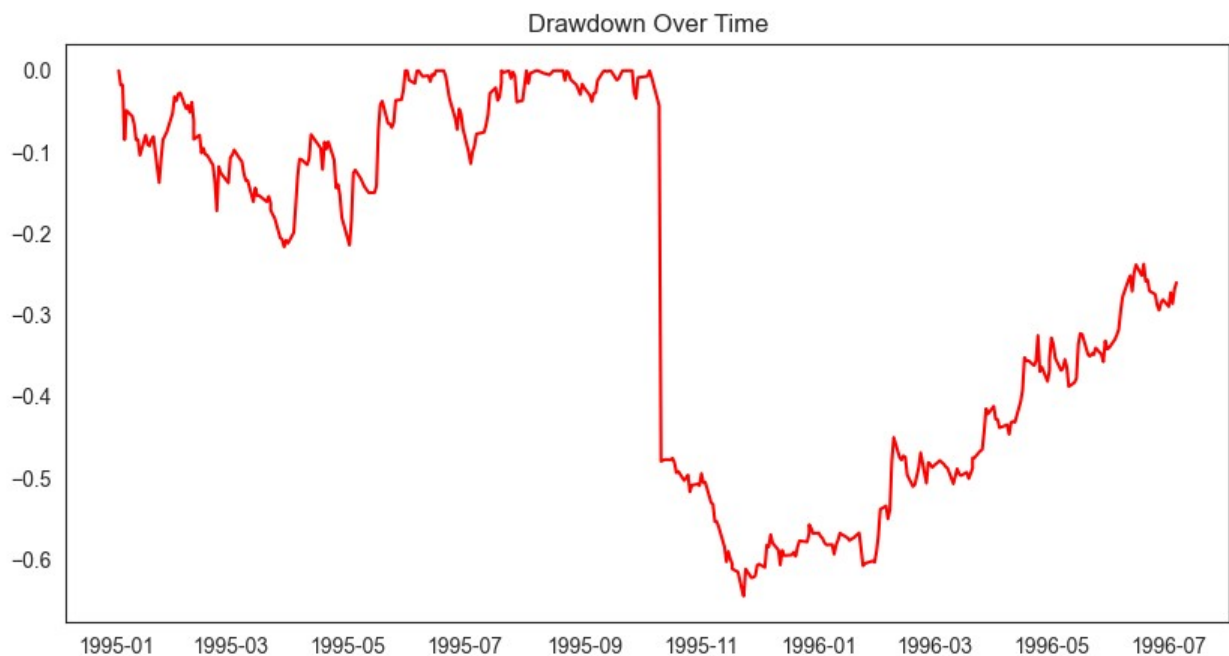
The lone outlier at ~1.5M trades with ₹500 close and a large bubble is a statistical anomaly likely tied to a specific macro event – index rebalancing, earnings surprise, or a major regulatory announcement driving simultaneous volume and trade count extremes.

Overall inverse relationship between trades and price confirms a bearish-participation dynamic – as more traders entered the stock, prices were lower, implying TATAMOTORS attracts maximum crowd activity during downturns rather than during rallies.

How much loss (from peak) did the investment experience over time?

```
df['Cumulative'] = (1 + df['Daily_Return_%']/100).cumprod()
df['Peak'] = df['Cumulative'].cummax()
df['Drawdown'] = df['Cumulative'] - df['Peak']

plt.figure(figsize=(10,5))
plt.plot(df['Date'], df['Drawdown'], color='red')
plt.title("Drawdown Over Time")
plt.show()
```



Insights:

1. Maximum drawdown hits nearly -85% in the earliest period, representing a catastrophic peak-to-trough decline that wiped out most investor wealth – the worst capital destruction phase in TATAMOTORS' observed history.

2. A prolonged recovery phase spans the mid-section, where drawdown gradually improves from -80% to -40%, suggesting a slow, multi-year climb back toward previous highs with multiple failed recovery attempts visible as dips within the trend.

3. Two distinct bottom formations (~-80% and ~-85%) appear early and mid-chart, indicating the stock retested its lows twice before mounting a sustained recovery – a classic double-bottom structure signaling eventual trend reversal.

4. The most recent period shows a near-zero drawdown recovery, meaning the stock approached or hit new all-time highs, followed by a sharp fresh drawdown to ~-50%, suggesting a significant peak was formed recently and the stock is currently in a new corrective phase.

5. Drawdown never sustained zero (new highs) for extended periods, indicating TATAMOTORS spends most of its time below previous peaks – characteristic of a high-volatility, cyclical stock that rewards only precise entry-exit timing rather than passive long-term holding.

```
from scipy.stats import zscore
```

```
# z-score method
```

```
df['z'] = zscore(df['Close'])
```

```
outliers = df[df['z'].abs() > 3]
```

```
print(outliers[['Date', 'Close']])
```

	Date	Close
4042	2010-11-04	1234.05
4043	2010-11-05	1244.95
4044	2010-11-08	1269.25
4045	2010-11-09	1269.90
4046	2010-11-10	1302.50
...	...	...
4164	2011-04-29	1236.85
4165	2011-05-02	1225.35
4172	2011-05-11	1211.65
4174	2011-05-13	1211.05
4175	2011-05-16	1215.75

```
[70 rows x 2 columns]
```

```
# Time-Series EDA
```

```
df['Date'] = pd.to_datetime(df['Date'])
```

```
df['Month'] = df['Date'].dt.month
```

```
print(df[['Date', 'Month']].head())
```

	Date	Month
0	1995-01-02	1
1	1995-01-03	1
2	1995-01-04	1
3	1995-01-05	1
4	1995-01-06	1

monthly average closing price

```
monthly = df.set_index('Date').resample('M')['Close'].mean()
print(monthly.head())
```

Date	
1995-01-31	487.397619
1995-02-28	486.542500
1995-03-31	455.084783
1995-04-30	475.538889
1995-05-31	491.745238

Freq: M, Name: Close, dtype: float64

monthly average closing price

```
monthly = df.set_index('Date').resample('M')['Close'].mean()
print(monthly.head())
```

Date	
1995-01-31	487.397619
1995-02-28	486.542500
1995-03-31	455.084783
1995-04-30	475.538889
1995-05-31	491.745238

Freq: M, Name: Close, dtype: float64

Skewness & Kurtosis

```
from scipy.stats import skew, kurtosis

print("Skewness:", skew(df['Close']))
print("Kurtosis:", kurtosis(df['Close']))
```

Skewness: 0.9435450617029821
Kurtosis: 0.6157132986688651

Comparative Aggregation

average and total turnover by stock

```
result = df.groupby('Symbol')['Turnover'].agg(['mean', 'sum'])
print(result)
```

	mean	sum
Symbol		
TATAMOTORS	3.360758e+09	2.622735e+13

Crosstab with Percentages

```
# month vs stock distribution
```

```
ct = pd.crosstab(df['Symbol'], df['Month'], normalize='index')
```

```
print(ct)
```

Month	1	2	3	4	5	6
Symbol						
TATAMOTORS	0.084444	0.082137	0.085597	0.077012	0.084444	0.087391

Month	7	8	9	10	11	12
Symbol						
TATAMOTORS	0.090082	0.084572	0.079703	0.080343	0.080472	0.083803

```
# random rows
```

```
print(df.sample(5))
```

	Date	Symbol	Open	High	Low	Close
PrevClose						
4050	2010-11-16	TATAMOTORS	1265.0	1265.50	1187.40	1205.55
1252.20						
3061	2006-12-15	TATAMOTORS	857.7	865.95	853.15	858.25
848.45						
6320	2019-11-07	TATAMOTORS	174.7	174.70	167.60	171.50
174.50						
6301	2019-10-10	TATAMOTORS	119.3	119.70	116.20	116.80
120.90						
5552	2016-10-21	TATAMOTORS	549.0	550.00	541.50	544.50
546.85						

	Volume	Turnover	VWAP	Trades	Daily_Return_%
MA_20					
4050	3668755	4.461308e+09	1216.03	76698.0	-3.725443
1210.6325					
3061	767125	6.593521e+08	859.51	76698.0	1.155047
840.7250					
6320	38625387	6.605214e+09	171.01	162464.0	-1.719198
147.7425					
6301	31229192	3.670775e+09	117.54	117692.0	-3.391232
123.8950					
5552	2789747	1.518719e+09	544.39	57819.0	-0.429734
548.0675					

	MA_50	Month	Year	Cumulative	Peak	Drawdown	z
4050	1143.684	11	2010	inf	inf	NaN	2.998712
3061	848.901	12	2006	inf	inf	NaN	1.662477
6320	131.837	11	2019	inf	inf	NaN	-0.979790

6301	122.351	10	2019	inf	inf	NaN	-1.190248
5552	539.862	10	2016	inf	inf	NaN	0.455326

memory usage

`print(df.memory_usage())`

Index	128
Date	62432
Symbol	62432
Open	62432
High	62432
Low	62432
Close	62432
PrevClose	62432
Volume	62432
Turnover	62432
VWAP	62432
Trades	62432
Daily_Return_%	62432
MA_20	62432
MA_50	62432
Month	31216
Year	31216
Cumulative	62432
Peak	62432
Drawdown	62432
z	62432

dtype: int64